

ELK stack

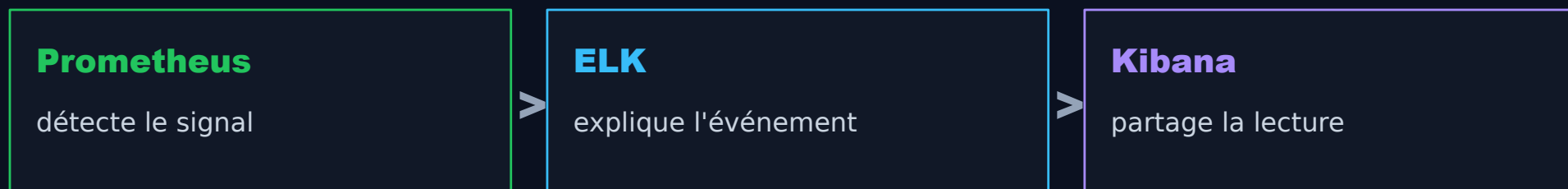


Elastic Search

Passer du signal à l'événement.

Cours 2 - logs, ingestion, recherche, dashboards
Kibana et usages développeur/support.

```
KQL
@timestamp >= "15:02" and
@timestamp <= "15:07"
and status >= 500
and service : "checkout"
```



Qui utilise ELK ?



Microsoft



La demande support commence souvent par une plage horaire.

Objectif du cours : savoir transformer une question floue en recherche exploitable.

"Retrouve-moi les logs entre 15h02 et 15h07."



Heure	Service	Message
15:03:12	checkout	payment refused
15:04:02	api	status 504
15:06:41	orders	retry ok

- Question temporelle : l'index doit porter un timestamp fiable.
- Question métier : route, fonctionnalité et identifiant anonymisé aident le support.
- Question technique : status, niveau, service et request_id aident les développeurs.

Prometheus trouve le symptôme ; ELK reconstruit l'histoire.

Les deux outils ne se remplacent pas : ils répondent à deux moments différents de l'investigation.

Signal métrique

- Combien d'erreurs ?
- Depuis quand ?
- Quelle tendance ?
- Quel seuil d'alerte ?

alerte 5xx



Événement journalisé

- Quelle requête ?
- Quelle route ?
- Quel message d'erreur ?
- Quel utilisateur ou client anonymisé ?

Métriques, logs et traces ne racontent pas la même chose.

Un bon diagnostic combine les trois au lieu de tout forcer dans un seul format.

Métriques

Combien ?

Séries temporelles
agrégées, alerting,
tendances, SLO.

Logs

Quoi exactement ?

Événements détaillés,
messages, contexte,
recherche temporelle.

Traces

Quel chemin ?

Parcours d'une requête
entre services, latence par
étape.

Le JSON structuré devient le contrat entre le code et Kibana.

La recherche commence au moment où le développeur choisit ce qu'il écrit sur stdout.

```
Erreur brute  
Payment failed for user 4812 on  
/checkout
```

```
Log structuré  
{  
  "level":"error",  
  "service":"checkout",  
  "route":"/checkout",  
  "status":504,  
  "request_id":"req-7fe1",  
  "feature":"payment"  
}
```

- Le brut se lit vite, mais se filtre mal.
- Le structuré permet les agrégations, les filtres et les dashboards.

Un log utile contient du contexte, pas des secrets.

L'enjeu développeur : rendre une erreur recherchable sans créer de risque sécurité ou RGPD.

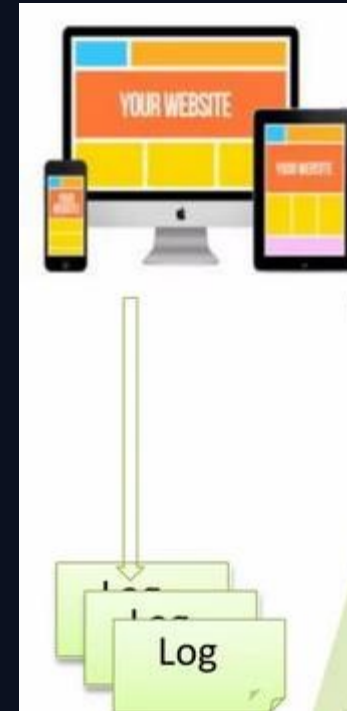
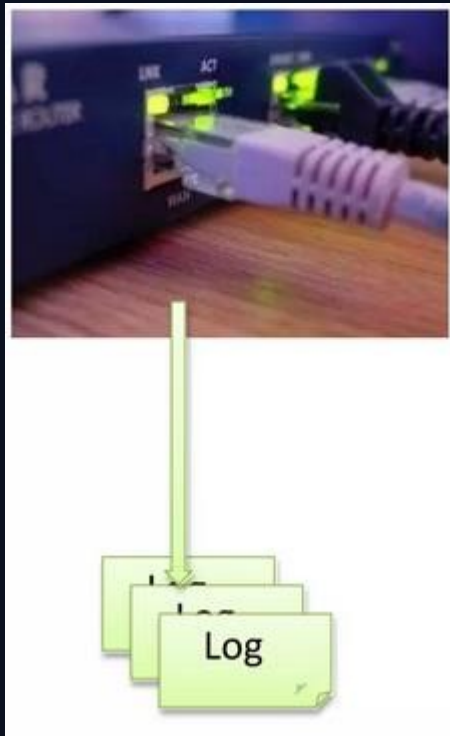
Champs à privilégier

- timestamp, level, service, environment
- route, status, status_family
- request_id ou trace_id
- message clair et durée
- champ métier anonymisé

À éviter

- tokens, mots de passe, cartes
- PII non anonymisées
- messages trop génériques
- logs debug permanents

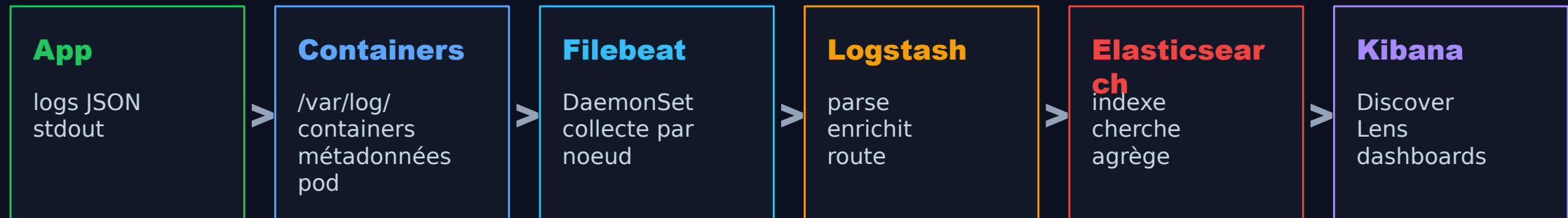
Dans Kubernetes, ELK part des logs stdout enrichis par le cluster.



```
bob@linux-ub:/var/www/html$ cat /var/log/apache2/error.log
[Sun Mar 27 20:27:00.269176 2016] [mpm_event:notice] [pid 23343:tid 140057225037696
] AH00489: Apache/2.4.7 (Ubuntu) configured -- resuming normal operations
[Sun Mar 27 20:27:00.269264 2016] [core:notice] [pid 23343:tid 140057225037696] AH0
0094: Command line: '/usr/sbin/apache2'
[Mon Mar 28 17:51:40.796855 2016] [mpm_event:notice] [pid 23343:tid 140057225037696
] AH00491: caught SIGTERM, shutting down
[Mon Mar 28 17:51:41.849873 2016] [mpm_event:notice] [pid 52009:tid 140535007307648
] AH00489: Apache/2.4.7 (Ubuntu) configured -- resuming normal operations
[Mon Mar 28 17:51:41.849978 2016] [core:notice] [pid 52009:tid 140535007307648] AH0
0094: Command line: '/usr/sbin/apache2'
[Mon Mar 28 17:53:49.477402 2016] [mpm_event:notice] [pid 52009:tid 140535007307648
] AH00491: caught SIGTERM, shutting down
[Mon Mar 28 17:53:49.530379 2016] [mpm_event:notice] [pid 53542:tid 140182453340032
] AH00489: Apache/2.4.7 (Ubuntu) configured -- resuming normal operations
[Mon Mar 28 17:53:49.530507 2016] [core:notice] [pid 53542:tid 140182453340032] AH0
0094: Command line: '/usr/sbin/apache2'
[Tue Mar 29 20:52:09.189540 2016] [mpm_event:notice] [pid 53542:tid 140182453340032
] AH00491: caught SIGTERM, shutting down
```


Dans Kubernetes, ELK part des logs stdout enrichis par le cluster.

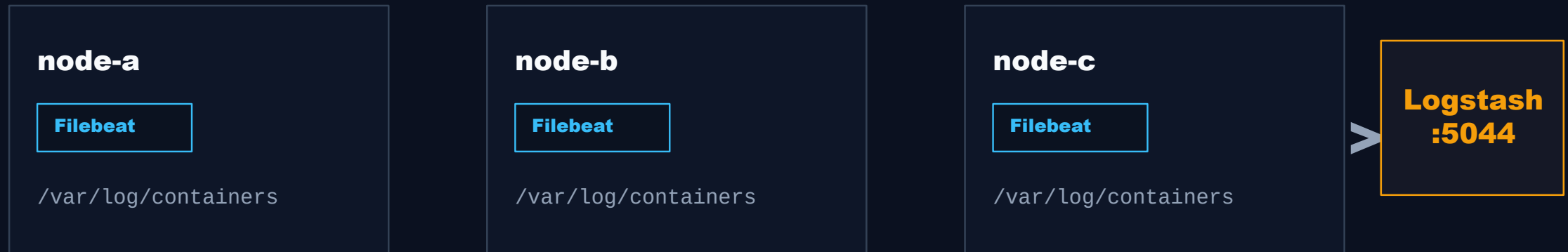
Le TP rend chaque étape visible avec des manifests simples.



Point pédagogique : chaque composant est déployé en YAML pour comprendre les objets Kubernetes réels.

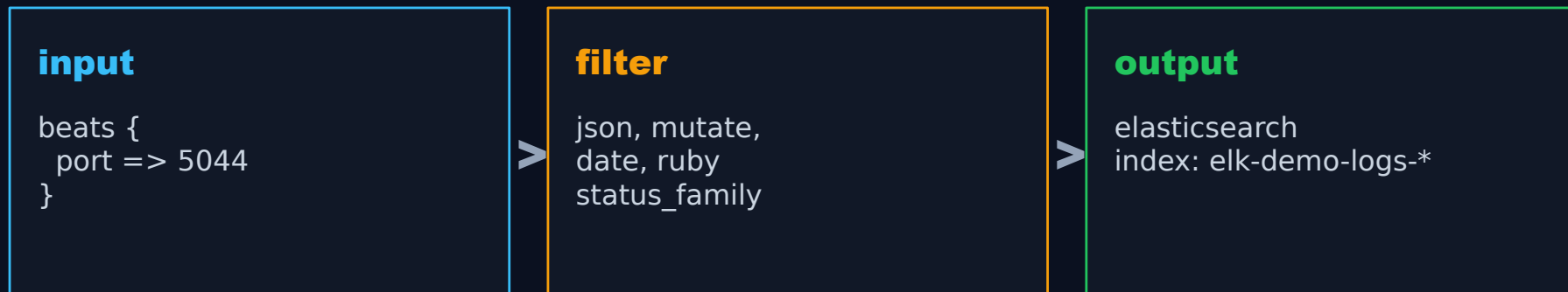
Filebeat suit les noeuds et ajoute le contexte Kubernetes.

Il ne connaît pas l'application : il lit les fichiers de logs des containers et ajoute les métadonnées utiles.



Logstash rend l'ingestion explicite : input, filter, output.

Pour un TP, il montre très bien où parser, enrichir et décider de l'index cible.



Exemple d'enrichissement
`status=504 -> status_family="5xx"`
`namespace="elk-demo" -> index TP dédié`

La qualité de recherche dépend des index et des champs.

Les bons champs évitent les requêtes lentes, ambiguës ou impossibles à agréger.

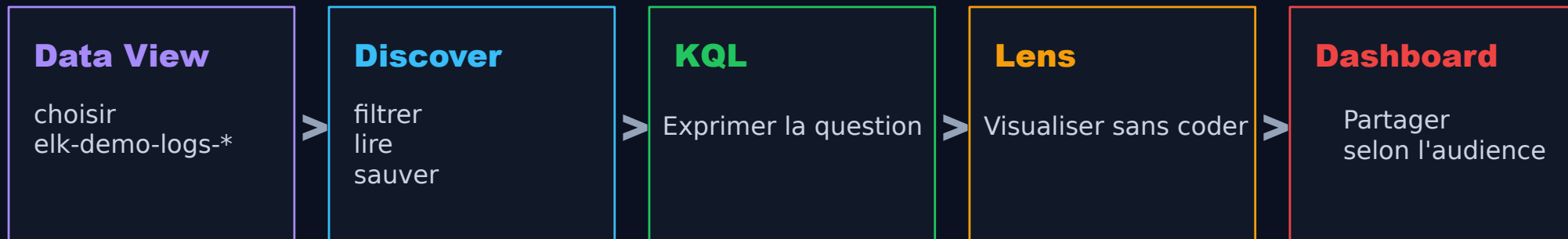
Notion	Rôle	Erreur classique
index	où les événements sont stockés	tout mélanger
mapping	type des champs	status en texte
keyword	filtrer / agréger	chercher sur text
timestamp	axe temporel	heure applicative absente

Exemple

```
status: 504
status_family: "5xx"
route.keyword:
"/checkout"
```

Kibana est l'interface d'investigation et de partage.

Discover sert à chercher ; Lens et Dashboard servent à stabiliser une lecture commune.



- Le développeur cherche les causes.
- Le support suit l'impact et les cas récents.
- Les deux exploitent les mêmes logs, mais pas les mêmes vues.

Le mode de déploiement choisit le niveau d'abstraction.

Le TP est manuel, mais les étudiants doivent situer Helm, ECK et le cloud.

Méthode	Avantage	Limite
Manifests	lisible, corrigeable	verbeux, maintenance manuelle
Docker Compose	excellent local	moins Kubernetes
Helm	standard, rapide	masque les objets
ECK Operator	robuste en prod	CRD et complexité
Elastic Cloud	opérationnel	moins formateur
OpenSearch	alternative à connaître	écosystème différent

Discover et KQL forment le cockpit d'investigation.

La requête doit rester proche de la question métier ou technique posée.

```
status >= 500 and route : "/checkout"  
level : "error" and service : "api"  
request_id : "req-7fe1"  
status_family : "4xx" or status_family : "5xx"
```

Routine

- borner le temps
- filtrer le service
- filtrer route/status
- ouvrir les logs liés
- sauver la recherche utile

Le dashboard développeur accélère le debug.

Il met au premier plan les causes probables : route, niveau, message, requête récente.

18

5xx sur 5 min
seuil alerte: 10

840 ms

p95 duration
/checkout

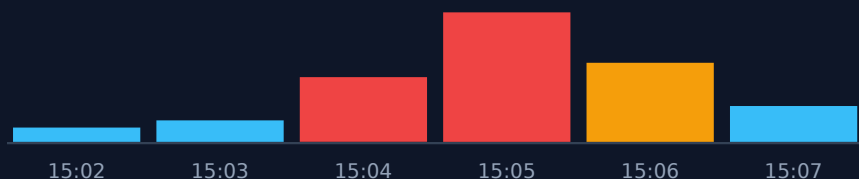
4

routes touchées
api + checkout

Top routes en erreur

/checkout		18
/orders		9
/login		5

Erreurs par minute



time	request_id	message
15:05:12	req-7fe1	payment timeout
15:05:19	req-8ab0	upstream 504
15:06:01	req-9cc2	validation failed

Dashboard | Views with system 3

Number of Events ECS

23,518
Count

TOP 10 Process.exe

TOP 10 event code

top 5 RDP connections

Number of Events Over Time By Channel ECS

Microsoft Windows...

Security

Application

System

Microsoft

Technique=Common

Microsoft-T1016, Tactic=Discovery

Note=WMF Querying (16.2%)

Microsoft-T1016, Tactic=Discovery

Microsoft-T1016, Tactic=Discovery

Microsoft-T1016, Tactic=Discovery

Microsoft-T1016, Tactic=Discovery

Microsoft-T1016, Tactic=Discovery

Microsoft-T1016, Tactic=Discovery

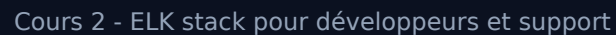
Microsoft-T1016, Tactic=Discovery

Microsoft-T1016, Tactic=Discovery

Process name .exe

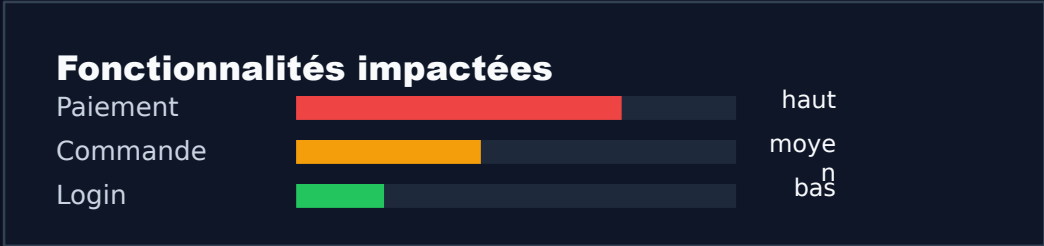
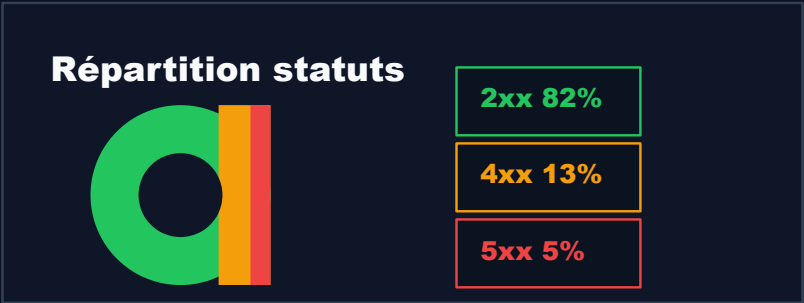
process.name: Descending	Count
cmd.exe	3,197
notepad.exe	1,636

IP-Port Destination



Le dashboard support traduit les logs en impact lisible.

Il évite les détails d'ingestion et met en avant les incidents récents, fonctionnalités et statuts.



heure	fonction	statut
15:05	Païement	dégradé
15:06	Commande	surveiller
15:07	Login	normal

Même donnée, décisions différentes.

Un dashboard n'est pas universel : il doit servir son audience.

Développeur

- identifier la cause probable
- filtrer par service, route, request_id
- lire les derniers messages erreur
- comparer avant/après déploiement

Support

- qualifier l'impact utilisateur
- suivre les incidents récents
- filtrer par fonctionnalité ou client
- anonymisé
répondre sans modifier l'ingestion

Les droits Kibana séparent exploitation et ingestion.

On peut donner de l'autonomie au support sans lui permettre de casser la collecte.



- Le support peut filtrer, lire et exporter une vue.
- Les développeurs gardent la responsabilité des logs produits par le code.
- L'équipe plateforme garde les pipelines, index et politiques de rétention.

Le TP valide une chaîne complète, pas une capture Kibana.

Les fichiers rendus doivent permettre de reconstruire la stack sur un cluster kind vierge.



Critère simple : une fois du trafic généré, Kibana doit permettre de retrouver, compter et visualiser les erreurs par famille de statut.

La baseline enseignant fixe les composants et les images.

Elle sert de référence de correction, pas de support à distribuer tel quel.

Composant	Objet K8s	Image
Elasticsearch	Deployment	elasticsearch:8.19.16
Kibana	Deployment	kibana:8.19.16
Logstash	Deployment	logstash:8.19.16
Filebeat	DaemonSet	filebeat:8.19.16
App demo	Deployment	elk-demo-app:1.0.0

Les logs deviennent dangereux quand on ignore volume, coût et données sensibles.

L'observabilité n'autorise pas à tout garder, tout indexer, ou tout exposer.

Volume

trop de debug, coût et bruit

Rétention

garder utile, purger le reste

Secrets

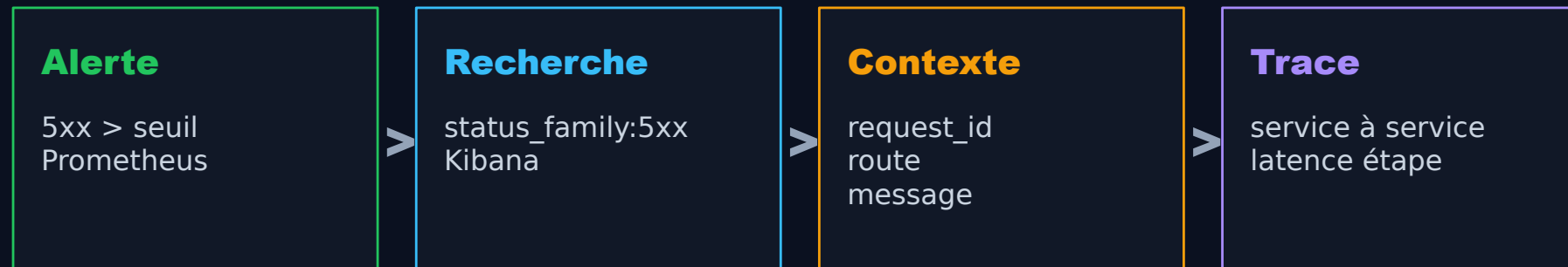
tokens et mots de passe interdits

Données perso

anonymiser, minimiser, tracer l'accès

La corrélation relie le signal, l'événement et le parcours.

Le `request_id` prépare naturellement le passage vers traces et APM.



`request_id="req-7fe1"` devient le fil rouge entre métrique, log et trace.

Prometheus détecte. Kibana explique.

- Une métrique dit qu'un comportement change.
- Un log donne l'événement précis.
- Un dashboard adapte la lecture à son audience.
- Un bon TP est redéployable depuis ses sources.

next: logs -> traces -> APM